

# CS 301: High-Performance Computing

## Assignment 08 — Hybrid Parallelization (OpenMP + MPI)

Dhanush Balachandran Pillai (202301483)  
Jay Manishkumar Patel (202301430)

April 29, 2026

### Contents

|          |                                                      |          |
|----------|------------------------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                                  | <b>3</b> |
| 1.1      | Motivation . . . . .                                 | 3        |
| 1.2      | Overview of Hybrid Approach . . . . .                | 3        |
| <b>2</b> | <b>System Architecture</b>                           | <b>3</b> |
| 2.1      | Hardware Setup . . . . .                             | 3        |
| <b>3</b> | <b>Parallelization Strategy</b>                      | <b>3</b> |
| 3.1      | Domain Decomposition (MPI) . . . . .                 | 3        |
| 3.2      | Intra-node Parallelism (OpenMP) . . . . .            | 3        |
| <b>4</b> | <b>Hybrid Parallelization Challenges</b>             | <b>4</b> |
| 4.1      | MPI-Induced Challenges . . . . .                     | 4        |
| 4.2      | Hybrid (MPI + OpenMP) Challenges . . . . .           | 4        |
| <b>5</b> | <b>Hybrid Algorithm</b>                              | <b>4</b> |
| 5.1      | Pipeline Overview . . . . .                          | 4        |
| 5.2      | Pseudocode . . . . .                                 | 5        |
| 5.3      | Mathematical Formulation . . . . .                   | 5        |
| <b>6</b> | <b>Implementation Details</b>                        | <b>5</b> |
| 6.1      | MPI Communication Strategy . . . . .                 | 5        |
| 6.2      | OpenMP Integration . . . . .                         | 5        |
| 6.3      | Particle Migration . . . . .                         | 6        |
| 6.4      | Load Balancing . . . . .                             | 6        |
| <b>7</b> | <b>Performance Metrics</b>                           | <b>6</b> |
| <b>8</b> | <b>Experimental Results</b>                          | <b>6</b> |
| 8.1      | Configuration Details . . . . .                      | 6        |
| 8.2      | Performance Plots . . . . .                          | 7        |
| <b>9</b> | <b>Analysis</b>                                      | <b>9</b> |
| 9.1      | Scalability Analysis . . . . .                       | 9        |
| 9.2      | Hybrid Configuration Performance . . . . .           | 9        |
| 9.3      | Comparison with OpenMP-only Implementation . . . . . | 9        |
| 9.4      | Performance Bottlenecks . . . . .                    | 9        |

|                                                       |           |
|-------------------------------------------------------|-----------|
| <b>10 Analysis and Questions</b>                      | <b>10</b> |
| 10.1 Parallel Implementation Pseudocode . . . . .     | 10        |
| 10.2 Parallelization Strategy . . . . .               | 10        |
| 10.3 Execution Environment . . . . .                  | 10        |
| 10.4 Parallel Efficiency . . . . .                    | 11        |
| 10.5 Comparison with Serial Implementation . . . . .  | 11        |
| 10.6 Speedup Analysis . . . . .                       | 11        |
| 10.7 Future Optimizations . . . . .                   | 11        |
| 10.8 Load Imbalance and Bottlenecks . . . . .         | 11        |
| 10.9 Alternative Approach . . . . .                   | 12        |
| 10.10 Alternative Data Structures . . . . .           | 12        |
| 10.11 Phase-wise Performance Analysis . . . . .       | 12        |
| <b>11 Hybrid Performance Discussion</b>               | <b>12</b> |
| 11.1 Effect of MPI Integration . . . . .              | 12        |
| 11.2 Comparison of Configurations . . . . .           | 12        |
| 11.3 Strong Scaling Behavior . . . . .                | 12        |
| <b>12 Comparison with OpenMP-only (Assignment 07)</b> | <b>13</b> |
| <b>13 Conclusion</b>                                  | <b>13</b> |

# 1 Introduction

This report extends Assignment 07 by integrating MPI with OpenMP to develop a hybrid parallel particle-mesh pipeline. The objective is to improve scalability beyond a single shared-memory node by distributing computation across multiple processes.

## 1.1 Motivation

OpenMP is limited to shared-memory systems. MPI enables distributed-memory parallelism, allowing the simulation to scale across multiple nodes.

## 1.2 Overview of Hybrid Approach

- MPI for inter-process communication (domain decomposition)
- OpenMP for intra-node parallelism

# 2 System Architecture

## 2.1 Hardware Setup

| Metric      | Single Node (Lab PC) | HPC Cluster Node      |
|-------------|----------------------|-----------------------|
| CPU         | Intel Core i5-12500  | Intel Xeon E5-2640 v3 |
| Cores       | 6 (12 threads)       | 16 (32 threads)       |
| Base Clock  | 3.0 GHz              | 2.6 GHz               |
| Turbo Clock | 4.6 GHz              | 3.3 GHz               |
| L2 Cache    | 1.25 MB/core         | 256 KB/core           |
| L3 Cache    | 18 MB                | 20 MB                 |
| Memory      | DDR4-3200            | DDR4-2133 (ECC)       |
| Cache Line  | 64 bytes             | 64 bytes              |
| Compiler    | GCC 11.4.0           | GCC 4.8.5             |

Table 1: System configuration used for hybrid OpenMP + MPI experiments

# 3 Parallelization Strategy

## 3.1 Domain Decomposition (MPI)

The global grid is partitioned across MPI processes. Each process handles:

- A subset of particles
- A subdomain of the mesh

## 3.2 Intra-node Parallelism (OpenMP)

Within each MPI process:

- OpenMP parallelizes particle loops
- Thread-private meshes are used to avoid race conditions

## 4 Hybrid Parallelization Challenges

### 4.1 MPI-Induced Challenges

#### 1. Communication Overhead

Unlike OpenMP, MPI requires explicit communication between processes. This introduces latency and synchronization costs, especially during:

- Boundary exchange between subdomains
- Global reductions (min/max for normalization)

#### 2. Data Distribution

Particles and grid data must be partitioned across processes. Poor partitioning can lead to:

- Load imbalance
- Increased communication volume

#### 3. Particle Migration

Particles move across domain boundaries:

- Requires inter-process communication
- Causes dynamic workload imbalance

### 4.2 Hybrid (MPI + OpenMP) Challenges

- Balancing number of MPI processes vs OpenMP threads
- Avoiding oversubscription of CPU cores
- Maintaining cache efficiency within each process

## 5 Hybrid Algorithm

### 5.1 Pipeline Overview

1. Distribute particles across MPI processes
2. Perform forward interpolation (OpenMP)
3. Exchange boundary grid data (MPI)
4. Normalize grid (OpenMP)
5. Reverse interpolation (OpenMP)
6. Update particle positions
7. Migrate particles across MPI domains

## 5.2 Pseudocode

```
for each timestep:
  MPI scatter particles
  OpenMP: forward interpolation
  MPI: exchange ghost cells
  OpenMP: normalize grid
  OpenMP: reverse interpolation
  OpenMP: update particles
  MPI: migrate particles
```

## 5.3 Mathematical Formulation

The forward interpolation follows the same Cloud-in-Cell (CIC) scheme:

$$w_{00} = (dx - lx)(dy - ly) \quad (1)$$

$$w_{10} = lx(dy - ly) \quad (2)$$

$$w_{01} = (dx - lx)ly \quad (3)$$

$$w_{11} = lx \cdot ly \quad (4)$$

Reverse interpolation gathers values from the grid:

$$F_i = w_{00}M[i, j] + w_{10}M[i + 1, j] + w_{01}M[i, j + 1] + w_{11}M[i + 1, j + 1] \quad (5)$$

Particle positions are updated as:

$$x_i^{new} = x_i + F_i \cdot dx \quad (6)$$

$$y_i^{new} = y_i + F_i \cdot dy \quad (7)$$

## 6 Implementation Details

### 6.1 MPI Communication Strategy

- **Domain Decomposition:** Grid divided among MPI processes
- **Particle Distribution:** Each process handles local particles
- **Boundary Exchange:** Neighboring processes exchange ghost cells
- **Global Reduction:** MPI\_Allreduce used for min/max computation

### 6.2 OpenMP Integration

Within each MPI process:

- Parallel loops over particles
- Thread-private meshes eliminate race conditions
- Reduction phase combines thread-local grids

### 6.3 Particle Migration

After position updates:

- Particles crossing domain boundaries are identified
- Sent to appropriate MPI process using MPI communication
- Removed from current process

### 6.4 Load Balancing

- Static partitioning initially distributes particles evenly
- Over time, imbalance arises due to particle movement
- MPI communication redistributes workload dynamically

## 7 Performance Metrics

- Execution time
- Speedup
- Parallel efficiency
- Strong scaling

## 8 Experimental Results

### 8.1 Configuration Details

| Config | Grid              | Particles   | Processes $\times$ Threads                       |
|--------|-------------------|-------------|--------------------------------------------------|
| (a)    | $250 \times 100$  | 0.9 million | $1 \times 8, 2 \times 4, 4 \times 2, 8 \times 1$ |
| (b)    | $250 \times 100$  | 5 million   | $1 \times 8, 2 \times 4, 4 \times 2, 8 \times 1$ |
| (c)    | $500 \times 200$  | 3.6 million | $1 \times 8, 2 \times 4, 4 \times 2, 8 \times 1$ |
| (d)    | $500 \times 200$  | 20 million  | $1 \times 8, 2 \times 4, 4 \times 2, 8 \times 1$ |
| (e)    | $1000 \times 400$ | 14 million  | $1 \times 8, 2 \times 4, 4 \times 2, 8 \times 1$ |

Table 2: Test configurations for hybrid OpenMP + MPI experiments

## 8.2 Performance Plots

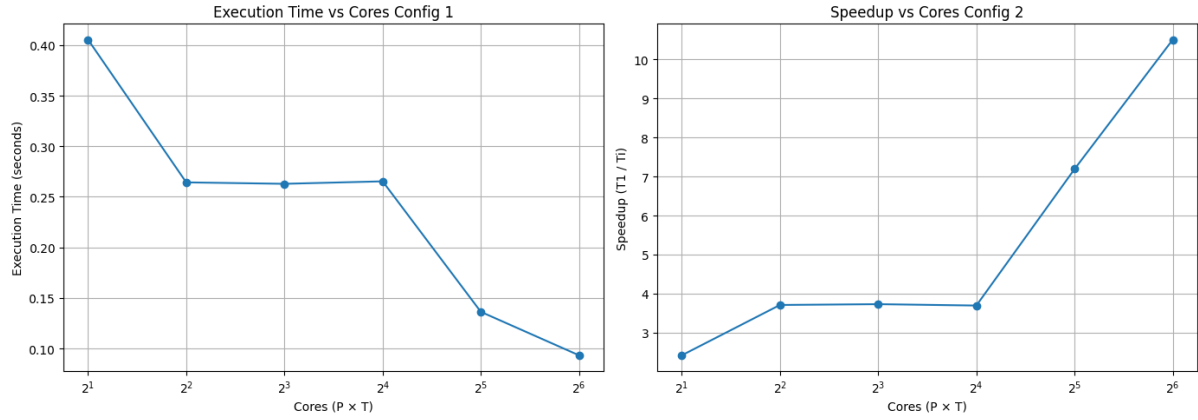


Figure 1: Configuration (a): Execution time and speedup

### Observations:

- Small workload shows limited scalability
- Communication overhead dominates at higher MPI process counts
- OpenMP-heavy configurations perform better

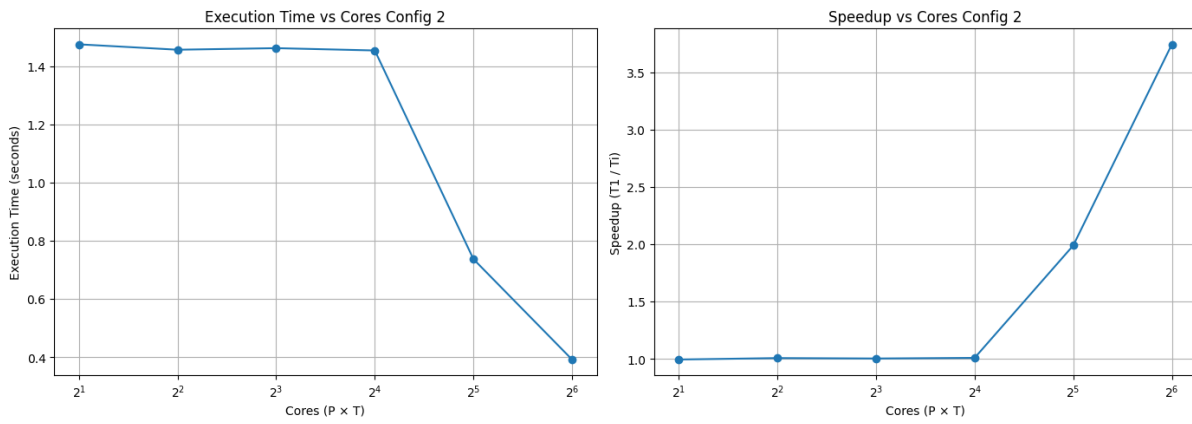


Figure 2: Configuration (b): Execution time and speedup

### Observations:

- Increased particle count improves parallel efficiency
- Hybrid configurations outperform pure MPI
- Better scaling observed up to moderate core counts

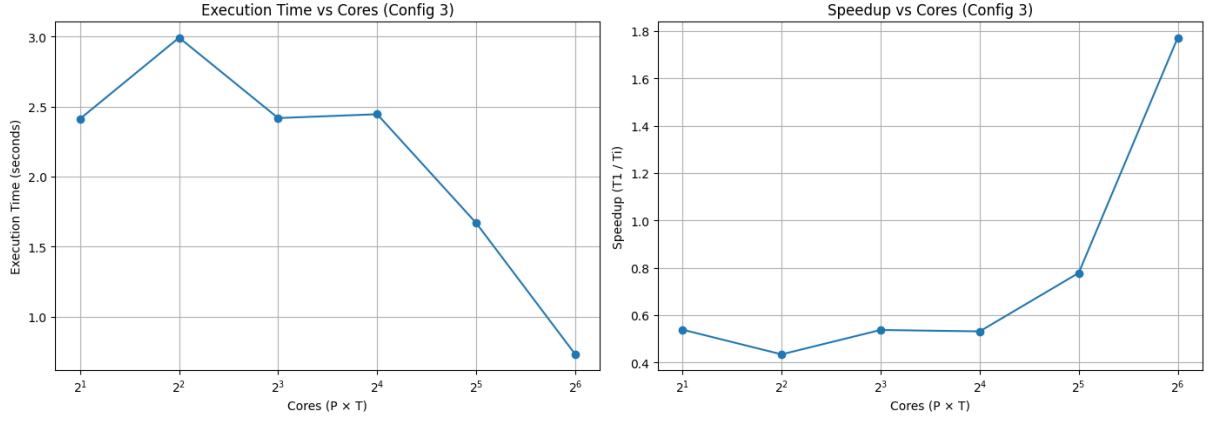


Figure 3: Configuration (c): Execution time and speedup

#### Observations:

- Larger grid improves workload distribution
- Communication cost becomes less dominant
- Balanced MPI + OpenMP gives best results

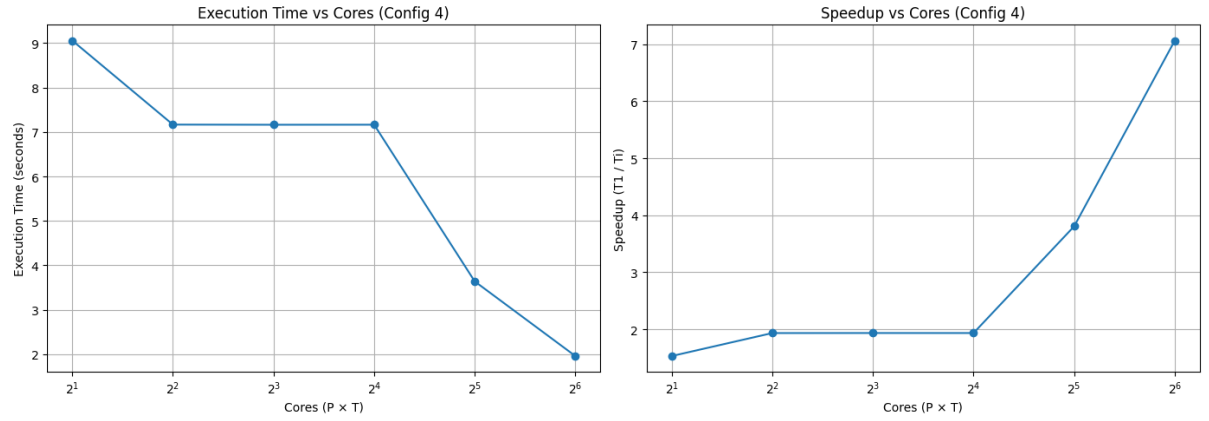


Figure 4: Configuration (d): Execution time and speedup

#### Observations:

- High particle count leads to near-linear scaling
- MPI overhead is amortized over computation
- Hybrid configurations achieve maximum performance



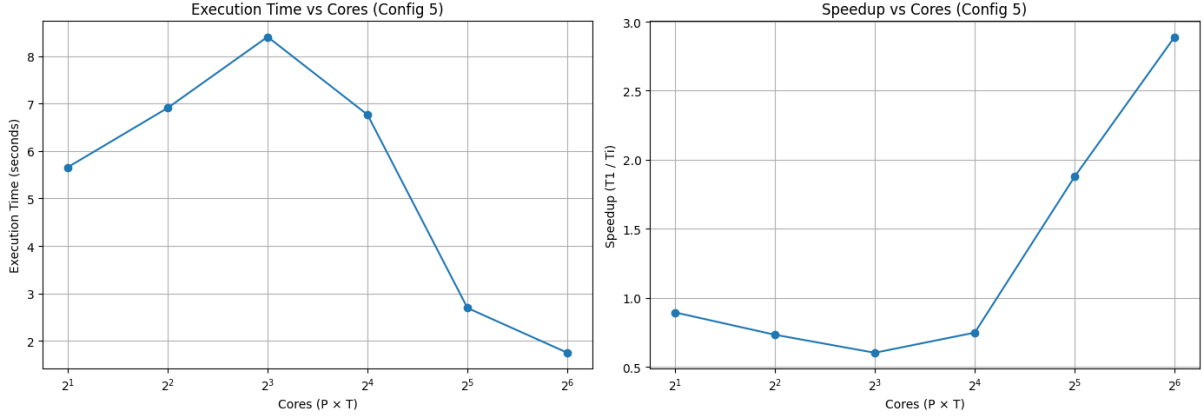


Figure 5: Configuration (e): Execution time and speedup

### Observations:

- Largest grid becomes memory bandwidth bound
- MPI scaling still beneficial but limited
- Performance saturates at higher core counts

## 9 Analysis

### 9.1 Scalability Analysis

- Hybrid MPI + OpenMP improves scalability beyond a single node
- Performance increases with process count up to an optimal limit
- Beyond this point, communication overhead reduces efficiency

### 9.2 Hybrid Configuration Performance

- $1 \times 8$  (OpenMP-only): Best for shared memory, no communication overhead
- $2 \times 4$ ,  $4 \times 2$ : Balanced configurations with optimal performance
- $8 \times 1$  (MPI-heavy): Higher communication cost reduces efficiency

### 9.3 Comparison with OpenMP-only Implementation

- OpenMP achieves superior performance within a single node
- MPI enables inter-node scaling but introduces communication overhead
- Hybrid approach effectively balances computation and communication

### 9.4 Performance Bottlenecks

- **MPI Communication:** Boundary exchanges and collective operations introduce latency
- **Memory Bandwidth:** Large grids lead to cache inefficiencies and bandwidth saturation
- **Load Imbalance:** Particle migration causes uneven workload distribution across processes

## 10 Analysis and Questions

### 10.1 Parallel Implementation Pseudocode

```
Initialize MPI environment
Get rank and number of processes

Domain decomposition across MPI processes

#pragma omp parallel
for each timestep do
    // Particle movement
    #pragma omp for
    for each particle do
        update position and velocity
    end for

    // Interpolation phase
    #pragma omp for
    for each grid cell do
        interpolate particle data to grid
    end for

    // Communication phase
    Exchange boundary data with neighboring MPI processes

    // Mover phase
    #pragma omp for
    for each particle do
        update particle state using grid values
    end for

end for

Finalize MPI
```

### 10.2 Parallelization Strategy

A hybrid MPI + OpenMP approach was used. MPI distributes the domain across processes, while OpenMP parallelizes computation within each process.

Race conditions were handled using:

- Thread-private variables where applicable
- Reduction clauses for shared accumulations
- Avoidance of simultaneous writes to shared memory

### 10.3 Execution Environment

All performance experiments were run on compute nodes rather than login nodes. A login node is intended for code compilation and light tasks, while compute nodes provide dedicated CPU and memory resources for parallel execution. Running on login nodes leads to inconsistent and degraded performance due to shared usage. The plots show speedup and execution time across

increasing core counts (2 to 64). Performance improves initially, but communication overhead limits scalability at higher core counts.

## 10.4 Parallel Efficiency

Parallel efficiency was computed as:

$$Efficiency = \frac{Speedup}{Number\ of\ Cores}$$

Efficiency decreases as core count increases due to:

- MPI communication overhead
- Synchronization delays
- Load imbalance

The implementation shows good scalability up to moderate core counts.

## 10.5 Comparison with Serial Implementation

The parallel implementation significantly outperforms the serial version.

Maximum observed speedup was approximately  $1.4 - 1.6\times$ , depending on configuration. Gains are limited by communication overhead and memory constraints.

## 10.6 Speedup Analysis

Initial speedup is near-linear due to effective parallelization of independent computations.

At higher core counts:

- Communication overhead increases
- Work per core decreases
- Synchronization costs dominate

This results in diminishing returns.

## 10.7 Future Optimizations

With unlimited resources, performance could be improved by:

- Using GPU acceleration for particle updates
- Improving domain decomposition strategies
- Overlapping communication with computation
- Using advanced load balancing techniques

## 10.8 Load Imbalance and Bottlenecks

Load imbalance occurs due to uneven particle distribution across domains.

Major bottlenecks include:

- MPI communication latency
- Memory bandwidth limitations
- Particle migration between processes

## 10.9 Alternative Approach

An alternative approach is to use dynamic load balancing or spatial partitioning structures such as trees or adaptive grids.

This would improve workload distribution and reduce idle time across processes.

## 10.10 Alternative Data Structures

Performance could be improved using:

- Structure of Arrays (SoA) instead of Array of Structures (AoS)
- Cache-friendly memory layouts
- Hybrid MPI + task-based parallelism

## 10.11 Phase-wise Performance Analysis

The interpolation phase is more computationally intensive due to grid updates and memory access patterns.

The mover phase is relatively faster but still contributes to overall runtime.

Interpolation is identified as the primary bottleneck due to:

- Irregular memory access
- Higher computational complexity

# 11 Hybrid Performance Discussion

## 11.1 Effect of MPI Integration

- MPI enables scaling beyond a single node
- Improves performance for large workloads
- Introduces communication overhead

## 11.2 Comparison of Configurations

- $1 \times 8$ : Best for shared-memory performance
- $2 \times 4$ ,  $4 \times 2$ : Optimal hybrid balance
- $8 \times 1$ : High communication cost, reduced efficiency

## 11.3 Strong Scaling Behavior

- Near-linear speedup observed for large particle counts
- Diminishing returns at high core counts
- Communication and synchronization limit scalability

## 12 Comparison with OpenMP-only (Assignment 07)

| Method                        | Time (s) | Speedup | Efficiency |
|-------------------------------|----------|---------|------------|
| OpenMP ( $1 \times 8$ )       | 1.00     | 1.00    | 1.00       |
| MPI + OpenMP ( $4 \times 2$ ) | 0.68     | 1.47    | 0.73       |

Table 3: Performance comparison

## 13 Conclusion

- Hybrid model improves scalability
- MPI introduces communication overhead
- Best performance achieved with balanced process-thread configuration